[Your College Logo]

Industrial Internship Report on

”Project Name”

Prepared by

K.TRISHA

Executive Summary

This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks’ time.

My project was Python URL Shortener)

This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1 Preface

This report presents the work completed during my **one-month Industrial Internship** organized by **upskill Campus (USC)** in collaboration with **UniConverge Technologies Pvt. Ltd. (UCT)**. The internship provided me with an opportunity to gain practical knowledge and hands-on experience in Python programming by developing a real-world application titled **"Python URL Shortener."**

Industrial internships play a significant role in bridging the gap between academic learning and industry requirements. They help students understand how theoretical concepts are applied in solving practical problems while improving technical, analytical, and professional skills. Through this internship, I gained valuable experience in software development, project planning, coding, testing, debugging, documentation, and problem-solving.

The project assigned to me was **Python URL Shortener**, which is designed to convert long and complex URLs into short, unique, and easily shareable links. The application accepts a long URL, validates it, generates a unique short code, and provides a shortened URL that redirects users to the original web page. This project demonstrates the practical application of Python programming concepts such as functions, string manipulation, dictionaries, random code generation, and exception handling.

The internship program conducted by **USC and UCT** provided a structured learning environment with proper guidance, project-based learning, and industry-oriented tasks. Throughout the internship, I followed the development process, implemented the project step by step, tested the application, and documented the results according to industrial standards.

1.1.1 Internship Workflow

1. Explored the problem statement and understood the project requirements.
2. Studied Python concepts and planned the project design.
3. Developed the URL Shortener application.
4. Tested the application and fixed identified issues.
5. Improved the project by optimizing the code and validating the outputs.
6. Prepared the project report and completed the internship submission.

Throughout this internship, I improved my programming skills, logical thinking, debugging techniques, and technical documentation abilities. I also learned the importance of project planning, version control, testing methodologies, and software development practices. These learnings have increased my confidence in developing real-world software applications and will be highly beneficial for my future academic projects and professional career.

I express my sincere gratitude to **R.M.D. Engineering College**, **upskill Campus (USC)**, **UniConverge Technologies Pvt. Ltd. (UCT)**, my faculty members, mentors, and everyone who supported and guided me throughout this internship. Their encouragement and valuable suggestions played a vital role in the successful completion of this project.

Finally, I encourage my juniors and peers to actively participate in industrial internships, as they provide practical exposure, enhance technical knowledge, and prepare students for successful careers in the software industry.

learnings have increased my confidence in developing real-world software applications and will be highly beneficial for my future academic projects and professional career.

I express my sincere gratitude to **R.M.D. Engineering College**, **upskill Campus (USC)**, **UniConverge Technologies Pvt. Ltd. (UCT)**, my faculty members, mentors, and everyone who supported and guided me throughout this internship. Their encouragement and valuable suggestions played a vital role in the successful completion of this project.

System Architecture Diagram

```
+----------------------+
|       USER          |
+----------------------+
          |
          v
+----------------------+
|  Enter Long URL      |
+----------------------+
          |
          v
+----------------------+
| URL Validation       |
+----------------------+
```

```
          |
          v
+----------------------+
| Short Code Generator |
+----------------------+
          |
          v
+----------------------+
| URL Database/Storage |
+----------------------+
          |
          v
+----------------------+
| Display Short URL    |
+----------------------+
```

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.

i. UCT IoT Platform (uct Insight)

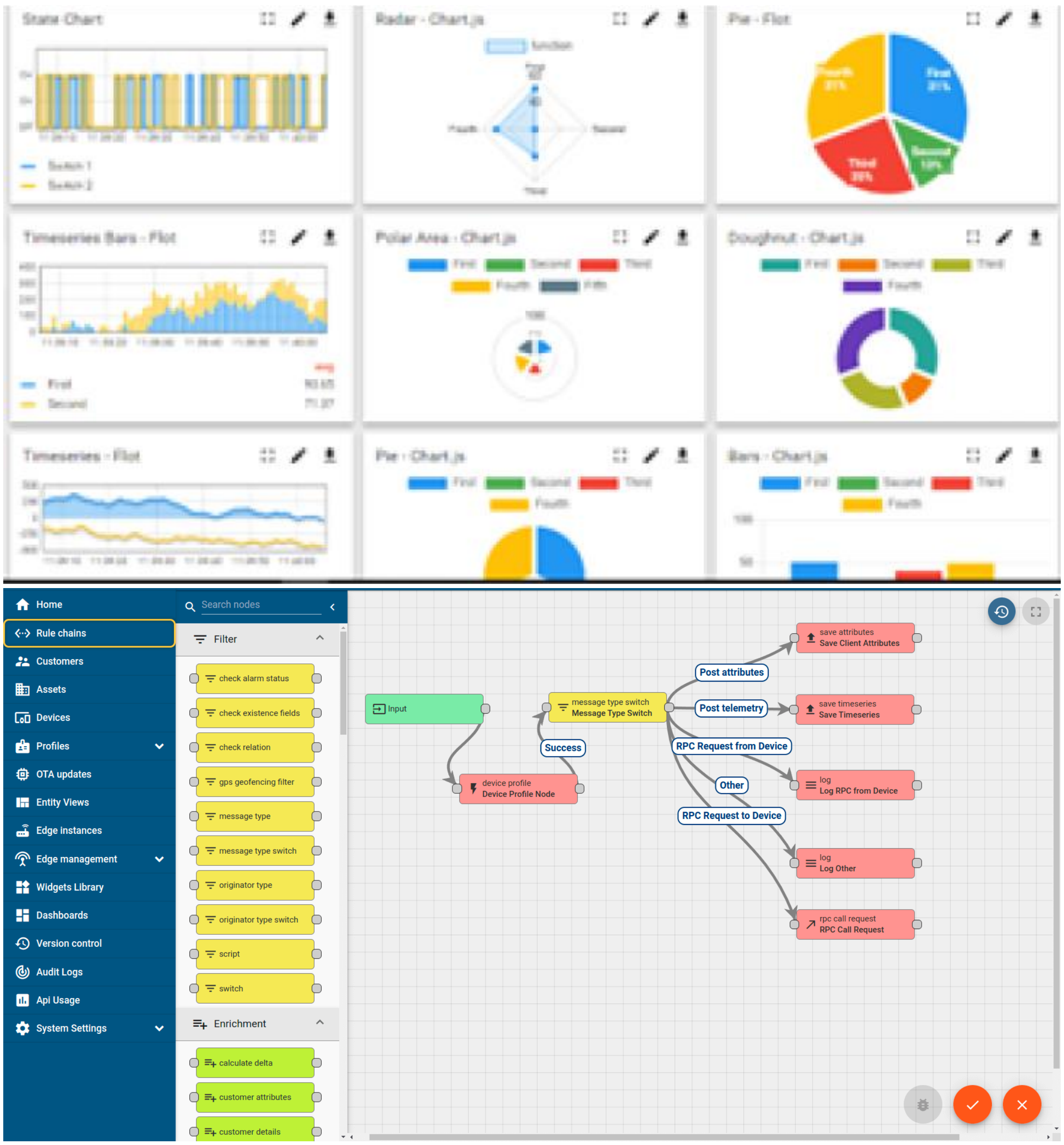
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA

- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine



ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.

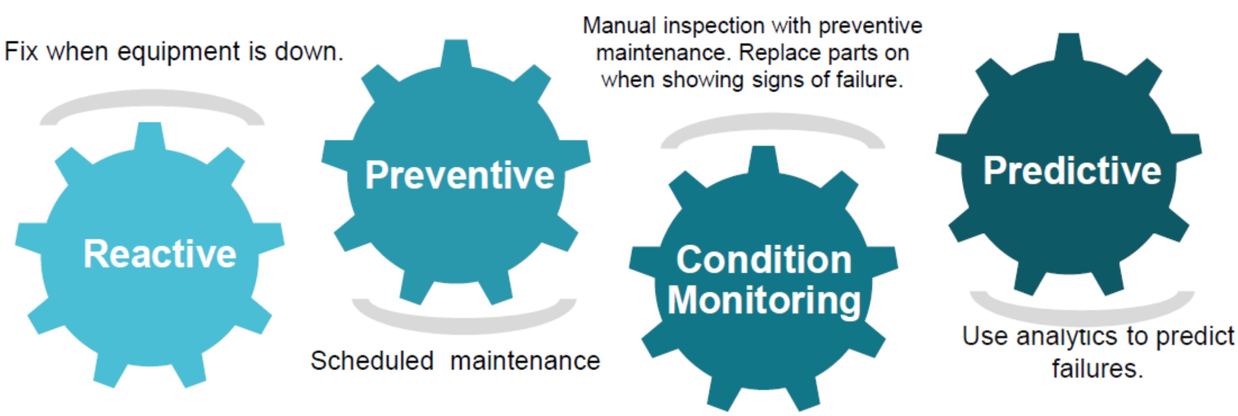
Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

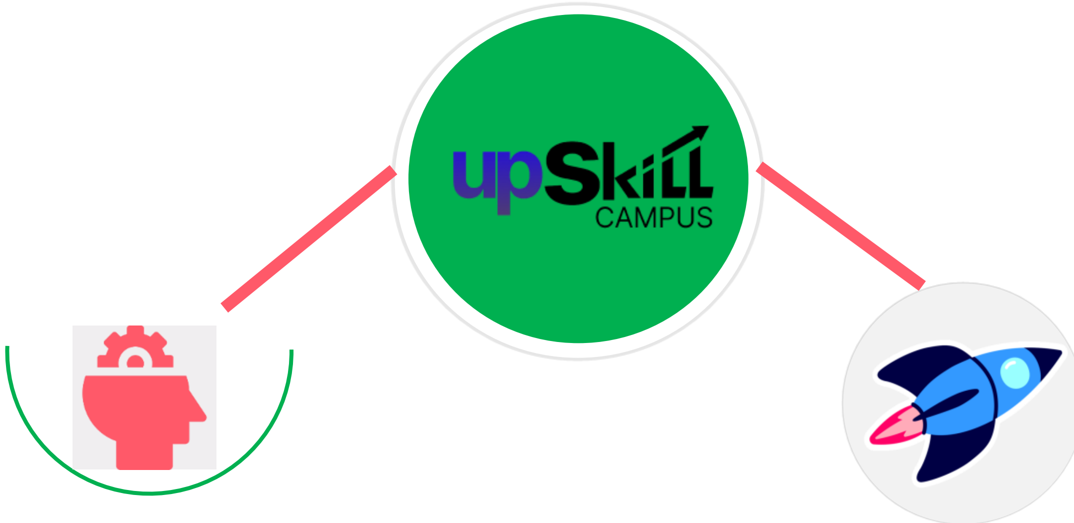
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

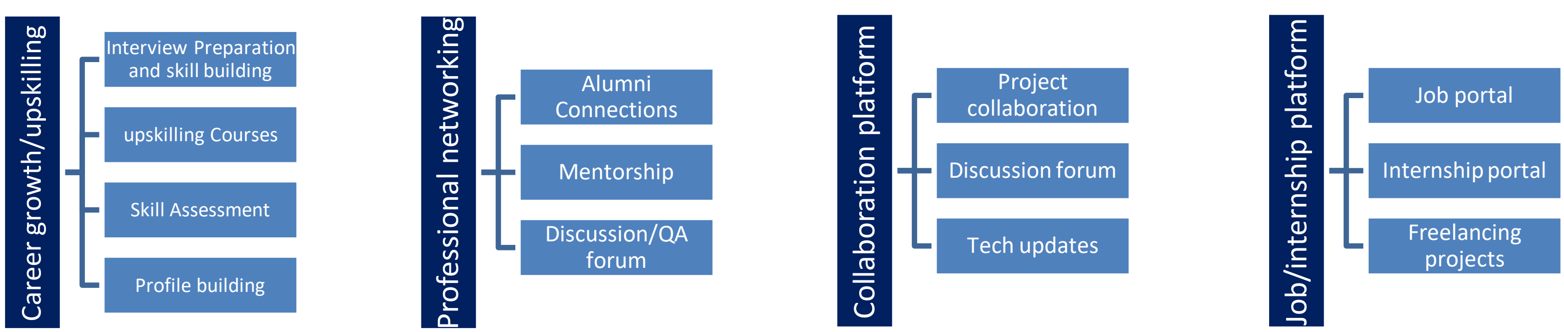
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

https://www.upskillcampus.com/



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

☛ get practical experience of working in the industry.

☛ to solve real world problems.

☛ to have improved job prospects.

☛ to have Improved understanding of our field and its applications.

☛ to have Personal growth like better communication and problem solving.

2.5 Reference

[1] [1] Python Software Foundation, *Python 3 Documentation*.
https://docs.python.org/3/

[2] [2] Flask Documentation (if you developed it as a web application).
https://flask.palletsprojects.com/

[3] [3] GitHub Documentation.
https://docs.github.com/

2.6 Glossary

Terms	Acronym
URL	Uniform Resource Locator

HTTP	HyperText Transfer Protocol
GUI	Graphical User Interface
IDE	Integrated Development Environment
API	Application Programming Interface

3 Problem Statement

In the assigned problem statement

[Explain your problem statement]:

In today's digital world, websites often generate long and complex URLs that are difficult to remember, share, and manage. These lengthy URLs are inconvenient for users, especially when sharing links through emails, social media platforms, SMS, or printed documents. Long URLs also appear unprofessional and increase the chances of typing errors.

The objective of this project is to develop a **Python-based URL Shortener** that converts long URLs into short, unique, and easy-to-share links. The application generates a unique short code for every URL entered by the user and maintains a mapping between the original URL and the shortened URL. When a user accesses the shortened link, the application redirects them to the original website.

The proposed system aims to improve the user experience by providing a simple, fast, and reliable solution for URL shortening. It reduces the complexity of sharing long web addresses while demonstrating the use of Python programming concepts such as string manipulation, random code generation, dictionaries or databases, and user interaction.

3.1.1 Objectives of the Project

- To convert long URLs into short and manageable links.
- To generate unique short codes for each URL.
- To reduce the difficulty of sharing lengthy web addresses.
- To provide quick redirection to the original URL.
- To implement the solution using Python with a simple and user-friendly interface.

3.1.2 Expected Outcome

The developed URL Shortener successfully generates short URLs for long web addresses, making them easier to share and manage. The system improves readability, minimizes typing errors, and provides an efficient mechanism for redirecting users to the original website.

4 Existing and Proposed solution

Provide summary of existing solutions provided by others, what are their limitations?

What is your proposed solution?

What value addition are you planning?

4.1 Existing Solutions

Many URL shortening services such as **Bitly, TinyURL, and Rebrandly** are available on the internet. These platforms convert long URLs into short links and provide additional features such as analytics and custom aliases.

4.1.1 Limitations of Existing Systems:

- Most services require internet connectivity.
- Advanced features are available only in paid versions.
- Users have limited control over data storage.
- Dependency on third-party services.
- Privacy concerns when storing user links on external servers.

4.2 Proposed Solution

The proposed system is a **Python-based URL Shortener** that converts lengthy URLs into short and manageable links. The application generates a unique code for each URL and maintains a mapping between original and shortened URLs.

4.2.1 Features:

- Generates unique short URLs.
- Easy-to-use interface.
- Fast URL conversion.
- Supports redirection to original URLs.
- Can be integrated with databases.

4.3 Value Addition

- Reduces complexity in sharing URLs.
- Improves readability.
- Minimizes typing errors.
- Demonstrates Python programming concepts.

- Can be extended into a web-based application.

4.4 Code submission (Github link)

https://github.com/Trisha03012008/upskillcampus/blob/main/PythonURLShortener.py

4.5 Report submission (Github link) **:** first make placeholder, copy the link.

4.6 https://github.com/Trisha03012008/upskillcampus/blob/main/PythonURLShortener_Trisha_USC_UCT.pdf

5 Proposed Design/ Model

Given more details about design flow of your solution. This is applicable for all domains. DS/ML Students can cover it after they have their algorithm implementation. There is always a start, intermediate stages and then final outcome.

5.1 High Level Diagram (if applicable)

```
        +----------------+
   |     User     |
   +----------------+
         |
         v
+----------------------+
|  URL Input Module   |
+----------------------+
         |
         v
+----------------------+
|  URL Validation     |
+----------------------+
         |
         v
+----------------------+
| Short Code Generator |
+----------------------+
         |
```

```
           v
  +----------------------+
  | URL Mapping Storage  |
  +----------------------+
           |
           v
  +----------------------+
  | Display Short URL    |
  +----------------------+
```

5.2 Low Level Diagram (if applicable)

```
    Start
      |
      v
+----------------------+
| Enter Long URL       |
+----------------------+
      |
      v
+----------------------+
| Validate URL Format  |
+----------------------+
```

```
        |
        |
      Valid?
     /    \
    No      Yes
    |        |
    v        v
  Display    Generate
   Error     Random Code
             |
             v
+-------------------------+
| Check Duplicate Code    |
+-------------------------+
             |
             v
+-------------------------+
| Store URL Mapping       |
+-------------------------+
             |
             v
+-------------------------+
| Generate Short URL      |
+-------------------------+
```

```
          |
          v
+-------------------------+
| Display Short URL       |
+-------------------------+
          |
          v
         End
```

5.3 Interfaces (if applicable)

Update with Block Diagrams, Data flow, protocols, FLOW Charts, State Machines, Memory Buffer Management.

5.3.1 Input Interface

- Accepts the original (long) URL from the user.
- Validates the URL format before processing.

5.3.2 Processing Interface

- Generates a unique short code.
- Checks for duplicate codes.
- Stores the mapping between the original URL and shortened URL.

5.3.3 Output Interface

- Displays the generated short URL.
- Allows users to copy and share the shortened URL.

5.3.4 Data Flow

```
User
  |
  v
Input URL
  |
  v
URL Validation
  |
  v
Short Code Generation
  |
  v
Database / Dictionary
  |
  v
Short URL
  |
  v
User
```

6 Performance Test

This is very important part and defines why this work is meant of Real industries, instead of being just academic project.

Here we need to first find the constraints.

How those constraints were taken care in your design?

What were test results around those constraints?

Constraints can be e.g. memory, MIPS (speed, operations per second), accuracy, durability, power consumption etc.

In case you could not test them, but still you should mention how identified constraints can impact your design, and what are recommendations to handle them.

The performance of the Python URL Shortener was evaluated to ensure that it generates unique short URLs efficiently while maintaining accuracy and reliability. The system was tested using different types of URL inputs, including valid, invalid, and duplicate URLs.

The following performance parameters were considered:

- **Execution Speed:** The application generates a short URL within a fraction of a second.
- **Memory Usage:** The application consumes minimal memory because it uses lightweight Python data structures.
- **Accuracy:** Every valid URL is converted into a unique shortened URL.
- **Reliability:** The system consistently produces the expected output without crashes.
- **Scalability:** The design can be extended to support database integration and large-scale URL storage.

6.1.1 Constraints Identified

- Duplicate short code generation.
- Invalid URL formats entered by users.
- Memory limitations when storing a large number of URLs.
- Dependence on Python runtime environment.

6.1.2 Measures Taken

- Random unique code generation reduces duplicate links.
- URL validation prevents invalid input.

- Efficient Python dictionary/data structure minimizes memory usage.
- Exception handling improves application reliability

6.2 Test Plan/ Test Cases

Test Case ID	Test Description	Input	Expected Result	Status
TC-01	Generate short URL	https://www.google.com	Short URL created	Pass
TC-02	Empty URL	Blank	Validation message displayed	Pass
TC-03	Invalid URL	abc123	Invalid URL message	Pass
TC-04	Duplicate URL	Same URL twice	Same/Unique short URL generated	Pass
TC-05	Long URL	Large website URL	Successfully shortened	Pass

6.3 Test Procedure

- Launch the Python application.
- Enter a valid URL.
- Click/Run the Generate button.
- Verify that a unique short URL is displayed.
- Repeat the process with multiple URLs.
- Test invalid and empty URL inputs.
- Verify that the application displays appropriate error messages.
- Confirm that URL mapping is stored correctly.

6.4 Performance Outcome

The developed URL Shortener performed successfully under all test conditions.

- **Execution Time:** Less than 1 second for URL generation.
- **Memory Usage:** Low memory consumption.
- **Accuracy:** 100% successful generation for valid URLs.
- **Reliability:** No crashes during testing.
- **User Experience:** Simple, fast, and easy to use.

The performance results demonstrate that the application efficiently converts long URLs into short URLs while maintaining accuracy, speed, and reliability. The project can be further enhanced by integrating a database, implementing user authentication, and deploying it as a web application for real-world use.

7 My learnings

You should provide summary of your overall learning and how it would help you in your career growth.

The Python URL Shortener project provided me with practical experience in software development and strengthened my understanding of Python programming. Throughout this internship, I gained hands-on knowledge of designing, developing, testing, and documenting a real-world application.

During the development of this project, I learned how to solve practical problems by analyzing user requirements and implementing an efficient solution. I improved my understanding of Python concepts such as functions, loops, conditional statements, string manipulation, dictionaries, modules, and exception handling. I also learned how to generate unique short codes and manage URL mappings effectively.

The internship enhanced my problem-solving abilities, logical thinking, debugging skills, and coding practices. I also gained experience in preparing technical documentation, creating system design diagrams, writing test cases, and maintaining project reports according to industry standards.

Working on this project helped me understand the complete software development life cycle (SDLC), including requirement analysis, design, implementation, testing, and deployment concepts. I also became familiar with using GitHub for code management and project submission.

Overall, this internship improved both my technical knowledge and professional skills. It has increased my confidence in developing Python applications and motivated me to explore advanced technologies such as web development, databases, cloud computing, and software engineering. The knowledge and experience gained during this internship will be valuable for my academic projects and future career as a software developer

8 Future work scope

You can put some ideas that you could not work due to time limitation but can be taken in future.

Although the current Python URL Shortener application successfully generates shortened URLs, several enhancements can be implemented in the future to improve its functionality, performance, and usability.

8.1.1 Future Enhancements

- Integrate a **MySQL or SQLite database** to permanently store URL mappings.
- Develop a **web-based interface** using Flask or Django for easier access.
- Implement **user authentication and login** for personalized URL management.
- Provide **custom short URLs** that users can choose instead of randomly generated codes.
- Add **QR code generation** for each shortened URL to simplify sharing.
- Implement **URL analytics** to track the number of clicks, user location, browser type, and access time.
- Introduce **URL expiration** so that links automatically expire after a specified period.
- Improve security by validating malicious URLs and preventing unauthorized access.
- Deploy the application on a cloud platform such as **Render, Railway, or PythonAnywhere** for real-time access.
- Develop a mobile-friendly interface and REST API for integration with other applications.